

作業二-演算法設計

⦿ Class Type	演算法
🕒 Created	2026年6月1日 上午1:14
⚙️ Status	未開始
☰ tag	114-2

智慧物流調度與路徑優化系統：演算法與系統設計規格書

1. 輸入 (Input) 規格定義

根據 express delivery 營運環境描述與限制條件，系統架構需完整接收並解析以下三種類型之資料輸入：

A. 城市拓撲地圖資料

- **圖形結構 $G=(V,E)$** ：包含頂點集合 V （代表各個城市或站點節點）與邊集合 E （代表站點間的實體聯絡道路）。
- **行車成本函數 $w(i,j) \rightarrow R^+$** ：權重代表車輛在兩相鄰頂點 i 與 j 之間移動所需的實質時間花費，而非單純的幾何距離。

B. 任務與目的地集合 ($V_{dest} \subseteq V$)

設定配送目的地集合 $V_{dest} = \{v_1, v_2, \dots, v_k\}$ ，其中每一目的地頂點 v_i 皆具備高度耦合之雙重業務屬性值：

- **送貨載重量 (d_i , Delivery Amount)**：定義為必須從起點配送中心（Depot）載運至該客戶節點的貨物數量。
- **取貨載重量 (p_i , Pickup Amount)**：定義為必須從該客戶節點攬收、並一路載回起點配送中心的逆向物流貨物數量。

C. 車隊資源與容量限制

- **車隊可用總數 (M , Number of Vehicles)**：系統可調配的最大合法車輛上限。

- **車輛最大載重上限 (C , Vehicle Capacity)**：全車隊每輛車一體適用之最大實體載貨容量限制（例如： $C = 40$ ）。
- **法定起訖點 ($s \in V$)**：所有被指派啟動的車輛，皆必須以中央配送中心 s 作為唯一的出發點與最終返回點。

2. 輸出 (Output) 規格定義

演算法求解完成後，系統需向調度介面與資料庫輸出符合最優化標準之解算結果：

A. 最佳車隊配送計畫 (Minimum Fleet Delivery Plan)

- 輸出一個能完全涵蓋 V_{dest} 中所有取送貨任務之多車巡迴路徑集合，並滿足車隊整體總移動時間最小化之目標：

$$\min \sum_{m=1}^M \text{TotalTime}(\text{Route}_m)$$

B. 精細車輛調度時間軸 (Detailed Delivery Schedule for Each Vehicle)

針對每一輛被實際派出的車輛 m ，輸出結構化之行程序列與動態狀態日誌：

- **拜訪序列 π_m** ：車輛 m 負責的站點先後順序，格式為：

$$(s, v_{\pi_m(1)}, v_{\pi_m(2)}, \dots, v_{\pi_m(h)}, s)$$

- **載重狀態軌跡**：車輛在各站點完成取送作業後之即時負載變化動態報告。

3. 核心演算法雙階段架構設計 (Two-Stage Algorithm Architecture)

本系統採用雙階段混合演算法架構：底層進行「全源最短路徑預處理」，高層基於完全圖矩陣執行精確解「分支界定搜尋法 (Branch-and-Bound Search)」。



Stage 1：底層成本計算 (Preprocessing)

- **Dijkstra's 全源成本矩陣化**：由於實體拓撲圖中節點間並非完全連接 (Fully Connected)，演算法首先以起點 s 及所有目的地 v_i 為源點，執行**戴克斯特拉演算法 (Dijkstra's Algorithm)**。

- **建構密集矩陣**：計算出任意兩點間的最短移動時間 $d(x,y)$ ，並建立一維度為 $(k+1) \times (k+1)$ 的成本矩陣（Cost Matrix）。高層演算法在分支尋路時，可以 $O(1)$ 的常數時間快速查表，徹底將「圖形尋路」與「調度排列」兩大問題域解耦。

Stage 2：高層路徑最佳化 (Branch-and-Bound Search)

透過自頂向下的解答樹空間搜索，尋找最優派車組合：

Step 2.1：初始化分支與界定邊界

1. 建立全域變數 U （Global Upper Bound），代表目前已找到之最優合法解的車隊總耗時，初始值設為無限大（ $U = \infty$ ）。
2. 建立一優先佇列（Priority Queue, PQ），依據路徑之「當前估計成本下界（Lower Bound）」進行最小堆疊（Min-Heap）排序，實現最佳優先搜尋（Best-First Search）策略。
3. 將根節點（狀態：所有車輛皆在起點 s ，未指派任何任務）推入 PQ 中。

Step 2.2：展開解答樹與動態狀態追蹤

只要 PQ 不為空，持續彈出（Pop）當前估計成本最小的搜索節點往下分支展開（每一分支代表指派當前車輛前往下一目的地 V_{next} ）。於每個搜索節點上，系統必須精確追蹤並封裝以下動態變數：

- `current_time`：該車輛當前已累積的行車時間。
- `route_delivery_sum`：關鍵控制變數。記錄該輛車此趟行程「已被指派的所有送貨任務量總和」。由於送貨必須在起點裝車，此數值即代表該車輛由配送中心出發時的初始負載。
- `current_load`：車輛在行駛路途中，完成當前站點取送作業後的即時車上貨量負載。

Step 2.3：容量限制的動態檢查與修剪規則 (Bounding & Pruning - Capacity)

此步驟為混合取送貨（VRPSD）的核心約束檢查。在嘗試將下一個目的地 v_{next} 納入當前車輛的拜訪序列時，必須即時通過雙重容量公式驗證：

1. **出發負載驗證**：檢查該路線累計之總送貨量，是否超出車輛實體上限：

$$route_delivery_sum + d_{next} \leq C$$

2. 沿途動態負載驗證：計算車輛到達下一站、完成交貨並執行取貨後的全新即時載重（`new_load`）：

$$\text{new_load} = \text{current_load} - d_{\text{next}} + p_{\text{next}}$$

1. 修剪判定（Termination Rule）：若以上任一條件不滿足（出發超載 $\text{route_delivery_sum} > C$ 或路途超載 $\text{new_load} > C$ ），判定該分支為不合法解（Infeasible Solution），立刻執行修剪（Prune），捨棄該解答子樹。

Step 2.4：時間成本的界定與修剪規則 (Bounding & Pruning - Time)

- 針對通過容量檢查的合法擴充路徑，演算法利用剩餘未拜訪節點的最小邊緣成本，快速估算該分支若走到底的總時間下界（Lower Bound, LB）。
- 修剪判定（Termination Rule）：若該分支的 $\text{LB} \geq U$ （全域最佳解），代表此路徑就算完成，其總時間也絕不可能優於現有已知最優解。演算法立刻執行修剪，阻止無效的深度搜索，大幅收斂計算複雜度。

Step 2.5：多車輛調度切換與邊界約束 (Multi-Vehicle Routing)

- 在解答樹分支時，演算法除了考慮「讓當前車輛繼續前往下一站」，亦同時展開平行分支——「當前車輛結單，由目前所在站點直接拉回起點 s ，並將其餘未完成任務指派給下一輛全新車輛從 s 出發」。
- 防呆邊界條件：若圖形中仍有未完成的剩餘任務，但系統可調配之車隊資源已達上限（即已被啟動的車輛數已等於 M ），則該「換車出發」的分支不合法，直接修剪。

Step 2.6：更新全域最佳解與終止輸出

1. 當演算法從 PQ 中彈出的節點滿足「所有 k 個任務目的地已全數分配完畢，且最後一輛車已成功拉回起點 s 」時，該節點即為一個全合法完整解（Feasible Solution）。
2. 讀取該解之車隊總耗時 F 。若 $F < U$ ，則更新全域最佳解 $U = F$ ，並將當前的車隊派車排程複製保存至最佳化紀錄中。
3. 當解答樹搜索完畢且 PQ 完全清空時，最終記錄在 U 中之調度方案即為全域絕對最佳解，系統正式向 UI 輸出最短配送計畫。